

MEDIATEK

MT8676 Reserved Memory User Manual

版本: 1.1
出版日期: 2024-11-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-10-28	李新	正式版
1.1	2024-11-12	李新	在章节 1.1 添加文档适用架构

目录

版本记录 2

目录 3

图片目录 3

1 Reserved Memory 4

1.1 什么是 Reserved Memory? 4

1.2 如何新增一块 Reserved Memory? 4

1.2.1 如何在 Kernel dts 中增加一个新的 Reserved Memory 节点? 5

1.2.2 如何使用 mblock 来 Reserve Memory? 5

1.3 Kernel Driver 如何获取 Reserved Memory 使用? 7

1.4 如何 Free 一块 Reserved Memory? 8

附件一 附加条款 10

图片目录

图 1-1. Reserved Memory 与 Memtotal 关系示意图 4

图 1-2. mblock_alloc 使用示例 6

图 1-3. mblock_query_reserved_by_name 使用示例 7

图 1-4. mblock_query_reserved_by_name 使用示例 8

图 1-5. mblock_free 使用示例 9

图 1-6. free_reserved_page 使用示例 9

1 Reserved Memory

1.1 什么是 Reserved Memory?

对于 Yocto 和 Android 来说，reserved memory 的意义和使用方法基本相同。此文档适用于 LA/LLA 架构。

简单来说，它主要是针对 Kernel 管理的内存资源，其中一部分会预留给 Kernel 系统自身运行或其他模块功能使用。预留的内存可以由特定模块独立使用，不受其他模块的影响。然而，这也会导致系统整体可动态分配的内存减少。

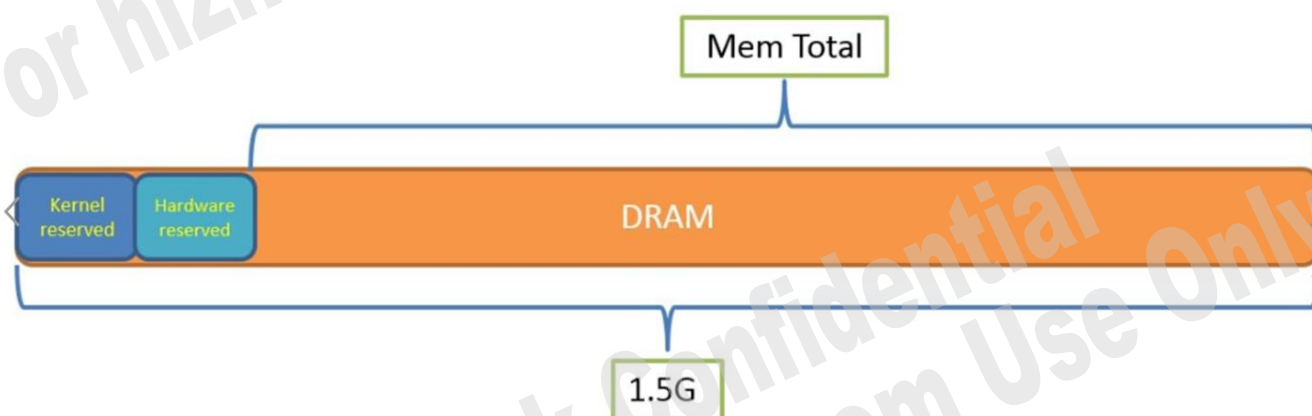


图 1-1. Reserved Memory 与 Memtotal 关系示意图

$\text{HW Dram size} = \text{MemTotal} + \text{HW Reserved Memory} + \text{Kernel Reserved Memory}$

Reserved Memory 主要包括 HW Module Reserved Memory 和 Kernel Reserved Memory 两部分：

1. HW Module Reserved Memory：这是为平台上的某些硬件或功能（如 TEE、SCP、显示等）预留的一部分内存，供其独立使用。
2. Kernel Reserved Memory：这是 Kernel 为存放其代码、数据以及关键功能的数据结构（如 struct page）和缓冲区而预留的内存。

1.2 如何新增一块 Reserved Memory?

当前，新增一块 reserved memory，有两种方法：

1. 在 Kernel dts 文件中增加一个新的 reserved memory 节点；
2. 在 LK2 中调用 mblock 的 API 来新增一块 reserved memory（推荐）。

1.2.1 如何在 Kernel dts 中增加一个新的 Reserved Memory 节点？

在 Kernel dts 中找到 reserved memory 节点，在这下面添加自己的 reserved memory 的子节点。

```
reserved-memory {
    xxx-reserved-memory {
        compatible = "xxx";
        no-map;
        size = <0 0x200000>;
        alignment = <0 0x200000>;
    };
    xxx-reserved-memory {
        compatible = "xxxx";
        reg = <0 0x44400000 0 0x10000>;
    };
    .....
};
```

- **蓝色字体**：reserved memory 的 start address;
- **红色字体**：size;
- alignment：对齐大小;
- compatible：关联相应的驱动程序，在 Kernel 启动流程阶段执行相应的驱动程序对这块 reserved memory 进行特殊处理;
- no-map：加上该属性意味着这块内存不会做 PA → VA 的线性映射，Kernel 也不会管理到这块 memory。

这种是 Kernel 原生提供的方法，更多节点含义可以参考

https://android.googlesource.com/kernel/msm/+android-7.1.0_r0.2/Documentation/devicetree/bindings/reserved-memory/reserved-memory.txt

1.2.2 如何使用 mblock 来 Reserve Memory?

mblock 是 MediaTek 开发在 LK2 阶段管理 reserved memory 的机制，新增一块 reserved memory 所使用的 API 为 mblock_alloc 和 mblock_alloc_range。

使用时需要指定以下参数：

参数	释义
reserved_size	预留的内存大小
align	预留内存的对齐大小
lower_bound	预留内存的地址下限
limit	预留内存的地址上限

参数	释义
expected_address	指定期望的 start address，如果是 free 的，则会分到这个 start address；如果已经被占用了会报错。不指定 expected_address 的话，会根据地址上下限动态地去找一块符合 size 要求的区间
mapping	预留内存的 mapping 类型，0 为 no-map，1 为 mapping，2 为 reusable
name	预留内存的名称
mblock alloc 的方向	默认会在 lower_bound~limit 范围内，从 limit 往前寻找，找到一块 free 的满足要求的 address 返回；如果找不到满足要求的，则会 fail.

使用示例如下：

```
/* allocate apusys-rv_secure */
if (need_relocate())
    sec_mem_addr_pa = mblock_alloc_range(sec_mem_size, APUSYS_SEC_MEM_ALIGN,
    APUSYS_MEM_LOWER_BOUND, APUSYS_MEM_UPPER_BOUND, 0, 0, "apu_apusys-rv_secure");
else
    sec_mem_addr_pa = mblock_alloc(sec_mem_size, APUSYS_SEC_MEM_ALIGN, APUSYS_MEM_LIMIT,
    0, 0, "apu_apusys-rv_secure");

if (!sec_mem_addr_pa) {
    dprintf(CRITICAL, "apu_apusys-rv_secure mblock allocation failed\n");
    ret = ERR_NO_MEMORY;
    goto err_apusys_rv_sec_alloc_fail;
}

/* set vmm */
ret = vmm_alloc_physical(vmm_get_kernel_aspace(),
    "vm-apu_apusys-rv_secure",
    sec_mem_size,
    &sec_mem_addr_va,
    PAGE_SIZE_SHIFT,
    (paddr_t)sec_mem_addr_pa,
    0,
    ARCH_MMU_FLAG_CACHED);

if (ret != NO_ERROR) {
    dprintf(CRITICAL, "APUSYS vmm_alloc_physical vm-apu_apusys-rv_secure failed, err: %d\n",
    ret);
    goto err_vmm_apu_apusys_rv_secure_alloc;
}

memset(sec_mem_addr_va, 0, sec_mem_size);

apusys_rv_initialize_sec_buf(sec_info_tmp, part_tmp_buf, sec_mem_addr_va, sec_mem_size);
```

图 1-2. mblock_alloc 使用示例

alloc 成功会显示如下 log，start: 0x8fa00000 为 alloc 的 start address。

```
mblock_alloc_range_no_lock:535: start: 0x8fa00000, sz: 0x600000 lower_bound: 0x0, limit:
0x90000000,mblock_alloc_range_no_lock:537: map:0 name:apu_apusys-rv_secure
```

补充:

1. 调用 `mblock_alloc` 分配 reserved memory 的原理与 dts 中的 reserved memory 是相同的。LK2 在结束时会将 `mblock_alloc` 分配的 reserved memory 信息以 dts 节点的形式写入 fdt 中, 对应的 log 为:

```
mblock_fdt_reserved_append:1595: mblock-reserved-memory is appended successfully
```

2. 在 LK2 其他文件中, 如果需要查询前面已经 reserved memory, 可以使用

`mblock_query_reserved_by_name(xxx, 0)`, 参数 xxx 是 reserved memory 的 name, 返回结构体 reserved, 其中 reserved->size 是查询到对应的 size, reserved->start 是查询到对应的 start address.

举例如下:

```
int aee_debug_kinfo_map(void)
{
    const struct reserved_t *reserved = NULL;
    status_t err;

    reserved = mblock_query_reserved_by_name("aee_debug_kinfo", 0);
    if (reserved) {
        err = vmm_alloc_physical(vmm_get_kernel_aspace(),
                                "aee_debug_kinfo", reserved->size,
                                &aee_debug_kinfo_va, 0,
                                reserved->start, 0, ARCH_MMU_FLAG_CACHED);
        if (err) {
            voprintf_error("vmm_alloc_physical for memmap fail %d\n", err);
            return -1;
        }
    } else {
        voprintf_error("query aee_debug_kinfo fail\n");
        return -1;
    }

    return 0;
}
```

图 1-3. mblock_query_reserved_by_name 使用示例

1.3 Kernel Driver 如何获取 Reserved Memory 使用?

在 Kernel driver 中通过 compatible 名称找到指定 node, 解析出对应 reserved memory 的 start phys address 和 size, 再映射成 virt address, 后续就可以使用这段 memory 了。举例如下:

```
if ((apu->platdata->flags & F_SECURE_BOOT) == 0) {
    apusys_sec_mem_node = of_find_compatible_node(NULL, NULL,
        "mediatek,apu_apusys-rv_secure");
    if (!apusys_sec_mem_node) {
        apu_drv_debug("DT,mediatek,apu_apusys-rv_secure not found\n");
        ret = -EINVAL;
        goto out_free_rproc;
    }
    of_property_read_u64_index(apusys_sec_mem_node, "reg", 0,
        &(apu->apusys_sec_mem_start));
    of_property_read_u64_index(apusys_sec_mem_node, "reg", 1,
        &(apu->apusys_sec_mem_size));
    apu_drv_debug("%s: start = 0x%llx, size = 0x%llx\n",
        apusys_sec_mem_node->full_name, apu->apusys_sec_mem_start,
        apu->apusys_sec_mem_size);
    apu->apu_sec_mem_base = memremap(apu->apusys_sec_mem_start,
        apu->apusys_sec_mem_size, MEMREMAP_WC);
}
```

图 1-4. mblock_query_reserved_by_name 使用示例

1.4 如何 Free 一块 Reserved Memory?

在 LK2 阶段 free 一块 reserved memory 所使用的 API 为 mblock_free 和 mblock_free_partial。

- int mblock_free(u64 addr); // 以 addr 起始的整块 reserved memory 全部 free
- int mblock_free_partial(u64 addr, u64 size); // free 起始地址为 addr，大小为 size 的部分的 reserved memory

使用时需要指定以下参数：

参数	释义
addr	整块 free 的话，这里填 alloc 时返回的 reserved memory 的 start address；部分 free 的话，这里填希望被 free 的部分 reserved memory 的 start address
size	要 free 的大小

使用示例如下：

```
void plat_memory_reserve_free(void)
{
    mblock_free(LK_KERNEL_64_BASE_PHY);
    mblock_free(LK_DT_BASE_PHY);
    mblock_free(LK_RAMDISK_BASE_PHY);
}
```

图 1-5. mblock_free 使用示例

free 成功有如下 log：

```
mblock_free_with_size:970: start 0x1c0000000 size: 0x7c00000, name: LK_KERNEL
```

注意：

当前，mblock 相关的函数 mblock_alloc 和 mblock_free 目前仅支持在 LK2 阶段调用。如果在 LK 阶段使用完 reserved memory，可以使用 mblock_free 来 free；如果需要保留到 Kernel 阶段使用一段时间再释放，可以在 Kernel 启动后调用 free_reserved_page 接口来 free。

举例如下：

```
static int free_reserved_buf(phys_addr_t start_phys, phys_addr_t end_phys)
{
    phys_addr_t pos;

    WARN_ON(start_phys & ~PAGE_MASK);
    WARN_ON(end_phys & ~PAGE_MASK);

    if (end_phys <= start_phys) {
        DDPPR_ERR("%s end_phys:0x%lx is smaller than start_phys:0x%lx\n",
            __func__, (unsigned long)end_phys, (unsigned long)start_phys);
        return -1;
    }

    for (pos = start_phys; pos < end_phys; pos += PAGE_SIZE)
        free_reserved_page(phys_to_page(pos));

    return 0;
}
```

图 1-6. free_reserved_page 使用示例

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。